Reporte: valores quemados en código — MediCentro

Oct 14, 2026 · @Fernando Guerra

Resumen

El front tiene IDs de catálogo escritos a mano que solo coinciden con la BD de quien los programó; en cualquier otra BD, Hospitalización, la bitácora de auditoría y Emergencias se rompen o muestran datos equivocados.

Se revisó el front `sistema_medicentro` (rama `Cambios-unificados`) y el backend `MediCentroBackendV3` (rama `main`), y se contrastó contra una BD local MEDICENTRO.

- **IDs de catálogo fijos:** 6 grupos, en unos 13 archivos. Es el problema más grave.
- **URL de la API fija:** `https://localhost:7086` aparece 69 veces en 48 archivos, sin archivo de configuración por ambiente.
- **Listas de ejemplo con IDs fijos:** 18 archivos `*-catalogos.ts` y formularios.
- **Backend:** el origen de CORS, la llave del JWT y la cadena de conexión están fijos. El backend sí resuelve los catálogos por código, así que ahí no hay IDs fijos.

IDs de catálogo fijos en el front

Estos números son `id_valor_catalogo` de una BD concreta; los IDs se generan con IDENTITY, así que cambian en cada BD. Rutas relativas a `sistema_medicentro/src/app/`.

Módulo	Archivo y línea	Valor quemado	Qué representa	En otra BD (ejemplo local)	Efecto
Hospitalización	`hospitalizacion/hospitalizacion-catalogos.ts` líneas 39–40	62, 63	Cama LIBRE / OCUPADA	LIBRE=220, OCUPADA=185 (el 62 es "Sala ocupada")	Nunca aparecen camas libres al crear una hospitalización; al guardar intenta poner a la cama un estado de sala
Hospitalización	`hospitalizacion-catalogos.ts` línea 49; `hospitalizaciones.service.ts` línea 152 (número directo)	65	Hospitalización ACTIVA	ACTIVA=187 (el 65 es "Martes")	"Agregar orden médica" dice que no hay hospitalizaciones activas aunque existan
Hospitalización	`hospitalizacion-catalogos.ts` líneas 52–55	68–71	Tipo de orden MÉDICA, ENFERMERÍA, MEDICACIÓN, ANESTESIA	190–193	Los accesos del menú que preseleccionan el tipo de orden apuntan a otro valor

Módulo	Archivo y línea	Valor quemado	Qué representa	En otra BD (ejemplo local)	Efecto
Hospitalización	`hospitalizacion/hospitalizacion-formulario/hospitalizacion-formulario.ts` líneas 198, 202, 398–407	usa las constantes 62, 63, 65	Filtro de camas libres y cambio de estado al guardar	—	Mismo efecto que las dos primeras filas
Hospitalización	`service/hospitalizacion.service.ts` línea 112	65	Copia duplicada del servicio anterior	ACTIVA=187	Mismo filtro roto en un segundo servicio
Auditoría (8 pantallas)	`laboratorio-lista.ts` 228–244, `orden-formulario.ts` 342–360, `hospitalizaciones-lista.ts` 237–253, `hospitalizacion-formulario.ts` 475–490, y 4 archivos de `emergencias/` (casos y compromisos)	79, 80, 81	Acción CREAR / MODIFICAR / ELIMINAR_LOGICO	143, 144, 145 (el 79 es un motivo de cancelación)	El historial muestra cada movimiento sin etiqueta ni ícono
Emergencias	`emergencias/emergencias-catalogos.ts` líneas 27 y 30; usado en `compromiso-formulario.ts` línea 192	46	Tipo de consentimiento "Compromiso de pago" (tabla `tipos_consentimiento`)	La tabla puede estar vacía; el 46 no existe	Guardar un compromiso de pago falla por llave foránea
Farmacia	`farmacia/recetas-lista/recetas-lista.ts` líneas 98–99	2, 3	Receta SURTIDA / CANCELADA	98, 99	Las recetas no toman el color de su estado
Farmacia	`farmacia/farmacia-catalogos.ts` líneas 21–24	1–4	Tipo de movimiento ENTRADA, SALIDA, AJUSTE, MERMA	157–160	Riesgo menor: busca por código y usa el número solo si falla; pero `movimiento-formulario.ts` líneas 58, 78 y 123 preselecciona SALIDA=2 directo

Todos se corrigen igual: pedir el ID por código a `/api/catalogos/{TIPO}`, como ya hacen Laboratorio y `farmacia/movimientos-lista.ts` (`idPorCodigo('TIPO_MOVIMIENTO_INVENTARIO', 'ENTRADA')`).

URL de la API fija

Cada servicio escribe su propia URL completa, `https://localhost:7086/api/...`: 69 apariciones en 48 archivos y no existe carpeta `src/environments`. Para apuntar a otro servidor, puerto o a http hay que editar los 48 archivos a mano.

Carpeta	Archivos con la URL
`service/`	43
`emergencias/`	2
`pacientes/`	1
`laboratorio/`	1
`hospitalizacion/`	1

Efecto práctico: el backend tiene que correr sí o sí con el perfil https en el puerto 7086; con el perfil http (5042) todo el front falla con "Failed to fetch".

Listas de ejemplo que siguen en el front

Varios formularios usan una lista de respaldo, con IDs 1, 2, 3…, cuando el catálogo no responde. Mientras el catálogo exista no se notan; si falta, el formulario guarda un ID que en la BD es otro valor o no existe.

Archivo	Qué tiene fijo	Riesgo
`consultas-externas/consultas-catalogos.ts`	`MEDICOS` (mock temporal) y estados de cita	Alto si se usa: médicos inventados
`laboratorio/laboratorio-catalogos.ts`	Tipos de examen, prioridades y estados con IDs 1–4	Medio
`mantenimiento/habitaciones-catalogos.ts`, `salas-catalogos.ts`, `empleados-catalogos.ts`, `aseguradoras-catalogos.ts`, `permisos-catalogos.ts`	Tipos y estados de habitación, sala y empleado; tipo de entidad; módulos	Medio: respaldo con IDs de ejemplo
`mantenimiento/interaccion-formulario.ts`, `interacciones-lista.ts`	Severidad con "los mismos ids de ejemplo que Pacientes"	Medio
`facturacion/facturacion-catalogos.ts`, `factura-formulario.ts`, `alertas/alertas-facturacion.ts`	Estados de factura y formas de pago	Medio
`farmacia/movimiento-formulario.ts`	Arranca con SALIDA=2 hasta que llega el catálogo	Bajo
`mantenimiento/convenio-formulario.ts`, `catalogos-lista.ts`	Respaldo si el catálogo no existe	Bajo

`permiso-formulario.ts` ya hace lo correcto: no usa respaldo, porque un ID de ejemplo puede coincidir con un valor de otro catálogo.

Valores fijos en el backend

El backend no tiene IDs de catálogo quemados: busca por código (`ResolverIdPorCodigoAsync("ESTADO_CAMA", "LIBRE")`). Lo fijo es configuración.

Archivo y línea	Valor	Problema
`appsettings.json` línea 6	Llave del JWT	Está en el repositorio desde el primer commit; cualquiera con acceso puede firmar tokens válidos
`appsettings.json` línea 3	Cadena de conexión con `Trusted_Connection=True`	Solo funciona con autenticación de Windows; en Mac o Linux cada quien la cambia y corre el riesgo de subir su contraseña
`Program.cs` línea 16	CORS: solo `http://localhost:4200`	Si el front corre en otro puerto o dominio, el navegador bloquea todo
`Servicios/CamasService.cs` líneas 14–15	Códigos `LIBRE` y `OCUPADA`	Correcto (es por código), pero el script de catálogos del equipo siembra `DISPONIBLE`, no `LIBRE`: hay que alinear el código

Aparte, el documento `SOLICITUD_ENDPOINTS_ELIMINADOS.md` propone código nuevo con `IdEstadoHospitalizacion == 65` y `IdEstadoCama = 62 / 63`. Conviene corregirlo antes de que alguien lo copie al backend.

Recomendaciones y prioridad

1. **Alta: Hospitalización.** Cambiar 62, 63, 65 y 68–71 por búsqueda por código. Eliminar el servicio duplicado `service/hospitalizacion.service.ts` o corregir ambos.

2. **Alta: Emergencias, compromiso de pago.** Buscar el tipo de consentimiento por `codigo = 'COMPROMISO_PAGO'` en lugar del 46, y sembrar esa fila en `tipos_consentimiento`.
3. **Alta: llave del JWT.** Sacarla del repositorio (User Secrets o variable de entorno) y generar una nueva, porque la actual ya quedó en el historial.
4. **Media: auditoría (79/80/81) y recetas (2/3).** Comparar por código (`CREAR`, `MODIFICAR`, `ELIMINAR_LOGICO`, `SURTIDA`, `CANCELADA`). Si el backend devuelve el código junto al ID, el front no necesita números.
5. **Media: URL de la API.** Crear `src/environments/environment.ts` con `apiUrl` y que los 48 servicios lo usen.
6. **Baja: listas de ejemplo.** Quitarlas, o mostrar un aviso en lugar de guardar IDs de ejemplo cuando el catálogo no carga.
7. **Baja: configuración local.** Mover la cadena de conexión y el origen de CORS a `appsettings.Development.json` o variables de entorno, para que cada quien use la suya sin tocar archivos versionados.

Regla para el equipo: ningún `id_valor_catalogo` escrito en código; siempre el par tipo + código.